

Aarya Arban

T11 – 05

Assignment No. 5

Aim: Connectivity of Node MCU with any sensor and show the output on serial monitor.

Theory:

The **NodeMCU** is an ESP8266-based microcontroller board with built-in Wi-Fi, making it suitable for IoT applications. In this experiment, we interface a **Flame Sensor** with the **NodeMCU** to detect fire and display the status on the **serial monitor**.

1. Understanding the Flame Sensor

A **flame sensor** detects fire based on the **infrared (IR) light** emitted by flames. It consists of an **IR receiver module** that identifies **specific IR wavelengths** and provides an output signal.

Flame Sensor Pin Configuration:

- **VCC (Power):** Connects to **3.3V or 5V** (NodeMCU supports 3.3V).
 - **GND (Ground):** Connects to **GND** on NodeMCU.
 - **DO (Digital Output):** HIGH (No flame detected), LOW (Flame detected). •
 - **AO (Analog Output - Optional):** Provides a flame intensity reading.
-

2. Working Principle of the System

- The **Flame Sensor detects fire** and sends a **digital signal** (HIGH/LOW) to the **NodeMCU**.
 - The **NodeMCU reads the sensor output** using a **GPIO pin**.
 - The **status (Flame Detected / No Flame)** is displayed on the **serial monitor** via `Serial.println()`.
-

3. Steps for Implementation

1. **Power the Flame Sensor** using **NodeMCU's 3.3V and GND**.
2. **Connect the DO (Digital Output) pin** of the Flame Sensor to a GPIO pin of **NodeMCU** (e.g., D5).
3. **Use the `Serial.begin(115200)` command** to initialize the serial monitor for output display.
4. **Continuously read sensor values** and print the status to the serial monitor.

Code:

```
int LED = D2;
int RELAY = D4;
int Flame_sensor_A = A0; // Analog pin for flame intensity
int threshold = 3; // Set your desired threshold for flame detection

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
  pinMode(RELAY, OUTPUT);
  digitalWrite(RELAY, HIGH);
  pinMode(Flame_sensor_A, INPUT);
}

void loop() {
  int flameIntensity = analogRead(Flame_sensor_A); // Read Analog
Value (0-1023)

  Serial.print("Flame Intensity: ");
  Serial.println(flameIntensity);

  if (flameIntensity > threshold) { // Below threshold means flame is
detected
    Serial.println("💧 Flame detected! Take action immediately.");
    digitalWrite(LED, HIGH); // Turn on LED
    digitalWrite(RELAY, LOW); // Turn off relay (could trigger a device
```

like a pump, etc.)

```
} else {  
  Serial.println("☑ No Flame detected. Stay cool.");  
  digitalWrite(LED, LOW); // Turn off LED  
  digitalWrite(RELAY, HIGH); // Turn on relay (to stay in safe state)  
}  
  
delay(1000); // Wait 1 second before the next reading  
}
```

Output:

```
☑ No Flame detected. Stay cool.  
Flame Intensity: 101  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 10  
☑ No Flame detected. Stay cool.  
Flame Intensity: 48  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 62  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 42  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 45  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 50  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 54  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 62  
🔥 Flame detected! Take action immediately.  
Flame Intensity: 53  
🔥 Flame detected! Take action immediately.
```

Conclusion:

In this experiment, we successfully interfaced a **Flame Sensor** with the **NodeMCU (ESP8266)** and displayed fire detection status on the **serial monitor**. This setup is useful for **fire detection systems, IoT-based security solutions, and home automation**. The **Wi-Fi capability of NodeMCU** allows future enhancements, such as sending alerts to a mobile application or cloud-based monitoring.